TALLER DE INTEGRACIÓN II

Probar endpoints, permisos y documentación Swagger

Informe de ejecución, resultados y problemas reproducidos

Proyecto Roomies - ¿Hay cupo? | Producto revisado: commit 63b8493

Se ejecutaron las pruebas existentes y nuevas matrices de permisos, contratos y documentación. Se añadieron cuatro verificadores reproducibles y evidencias JSON. No se modificó la lógica de la aplicación ni se escribieron usuarios, imágenes o avisos reales.

Comprobación	Casos	PASS	FAIL	INFO
Suite Python existente	53	53	0	0
Permisos y cierre de sesión	55	40	15	0
Contratos y robustez	29	20	9	0
Swagger / OpenAPI	28	20	6	2
HTTP local y CDN de documentación	11	11	0	0

La suite JavaScript también pasó. Las matrices adicionales suman **123 comprobaciones: 91 PASS, 30 FAIL y 2 INFO**. Varias filas detectan la misma causa: no equivalen a 30 vulnerabilidades distintas.

Veredicto

Pruebas terminadas; sistema no aprobado para producción. Los flujos normales y los filtros de dueño funcionan, pero la identidad se puede suplantar y el acceso privado permanece disponible sin sesión. Hay fallos de robustez y documentación que también deben corregirse.

Un FAIL compara el resultado con una política segura o robusta. Puede ser una limitación conocida de la demostración, no una regresión reciente. Se conservaron las expectativas y los resultados negativos; no se ajustaron para producir un informe completamente verde.

02 / PRUEBAS Y EVIDENCIAS

Método, aislamiento y archivos entregados

Las rutas reales de publicaciones y autenticación se cargaron con el router de conexiones sustituido. Las matrices usan TestClient, conexiones falsas que aplican dueño y archivos simulados. Se bloqueó psycopg.connect y se sustituyó load_dotenv para impedir el uso de PostgreSQL o credenciales reales.

El smoke HTTP utilizó un servidor Uvicorn temporal en 127.0.0.1, con puerto libre y lifespan desactivado. Se consultaron documentos, páginas y recursos estáticos; el servidor quedó detenido al terminar. Los tres recursos públicos del CDN se descargaron solo para comprobar disponibilidad, sin ejecutar JavaScript.

Archivo / carpeta	Contenido
scripts/ **verificar_permisos_publicaciones.py**	55 casos de identidad, dueño, anónimo, cookie inválida/forjada y cierre de sesión.
scripts/ **verificar_contratos_publicaciones.py**	29 casos de errores, archivos, datos antiguos, recursos inexistentes y proveedor geográfico simulado.
scripts/ **verificar_swagger_publicaciones.py**	28 criterios de HTML, OpenAPI, esquemas, seguridad y coherencia de errores.
scripts/ **verificar_http_swagger.py**	8 comprobaciones por TCP local y 3 de recursos del CDN.
output/qa/ **permisos.json / contratos.json**	Cada solicitud, resultado esperado/obtenido y escrituras o bytes simulados.
output/qa/ **swagger.json / openapi_auditado.json**	Criterios y copia del esquema auditado de publicaciones/auth.
output/qa/ **http_swagger.json / resumen.json**	Resultados HTTP y consolidación de esta revisión.
output/qa/ **LEEME.md / unittest_log.txt**	Comandos, límites y log de las 53 pruebas Python.

Las identidades y registros son ficticios: publicaciones 42 del autor 1, 43 del autor 2 y 44 del autor 37. Cada caso aislado reinicia los datos para que un borrado no altere otra comprobación. El flujo login/logout sí conserva la cookie y los datos entre sus pasos.

03 / PRUEBAS Y EVIDENCIAS

Permisos: matriz ejecutada

Se distingue **comprobar al dueño en SQL** de **autenticar a la persona**. Las cookies numéricas permiten probar el filtro de dueño, pero no representan sesiones verificadas. La expectativa de seguridad es 401 para una operación privada sin una sesión válida.

Actor / situación	Casos	PASS / FAIL	Resultado observado
Identidad declarada 1	11	11 / 0	Operaciones públicas y propias responden como se esperaba en la fixture.
Identidad declarada 2	11	11 / 0	Puede operar sobre sus datos; editar, cerrar o borrar el aviso 42 devuelve 404 sin escrituras ajenas.
Sin cookie	11	4 / 7	Lo público funciona. Las siete operaciones privadas devuelven 200/201 usando usuario 1 en lugar de 401.
Cookie no numérica abc	11	11 / 0	Lo público funciona; las siete operaciones privadas responden 401 antes de consultar la base.
Cookie forjada 37	7	0 / 7	Las siete operaciones privadas aceptan ese ID sin comprobar una sesión.
Recurso inexistente	3	3 / 0	PUT, PATCH y DELETE del aviso 999 responden 404.
Login y logout	1	0 / 1	Después de eliminar la cookie, vuelve a acceder como usuario 1.

Operaciones privadas cubiertas

```
GET    /api/mis-publicaciones
POST   /api/publicaciones
PUT    /api/publicaciones/{id}
PATCH  /api/publicaciones/{id}/cerrar
DELETE /api/publicaciones/{id}
GET    /api/perfil
PUT    /api/perfil
```

Listado, búsqueda, detalle y consulta de cupos se consideran públicos en esta matriz y se comprobaron con y sin cookie. Su acceso anónimo es un PASS, no una omisión de seguridad.

04 / PRUEBAS Y EVIDENCIAS

Fallo principal: identidad y cierre de sesión

1. Sin sesión se usa el usuario 1

Referencia: main/main.py:89, autor_de_prueba. El parámetro Cookie tiene default="1" y los números positivos se aceptan como identidad. Sin cookie, las operaciones privadas usan al autor 1; cambiar el número suplanta otro ID. El filtro SQL de dueño no impide esta suplantación.

Comprobación	Esperado seguro	Obtenido
Publicar sin cookie	401; sin escritura	201; autor simulado 1.
Editar/cerrar/borrar aviso 42 con ID declarado 2	404; sin cambio ajeno	404; sin escrituras ajenas: correcto.
Operar con cookie forjada 37	401; identidad no validada	200/201; opera como autor 37.

2. Logout elimina la cookie, pero no elimina el acceso privado

```
Login usuario QA 2 -> 200
GET propias -> 200; devuelve aviso 43
POST /auth/logout -> 200; cookie eliminada
GET propias sin cookie -> 200; devuelve aviso 42 del usuario 1
Esperado seguro al terminar -> 401
```

La llamada de logout y la eliminación de la cookie funcionan. El fallo está en el respaldo del backend al usuario 1: al salir se recupera acceso privado bajo esa identidad. La secuencia se probó con las rutas reales de login/logout y una base simulada.

Qué debe cambiar

Obtener el autor de una sesión verificable, eliminar el respaldo automático y rechazar a quien no tenga sesión válida. Después debe declararse el mecanismo de seguridad en OpenAPI. Añadir un candado o un campo de cookie en Swagger por sí solo no autentica al usuario.

El código conserva autenticación de demostración y comparación directa de contraseña en SQL. Esa observación de implementación debe revisarse antes de utilizar cuentas reales; no se migraron cuentas ni se cambiaron contraseñas durante estas pruebas.

05 / PRUEBAS Y EVIDENCIAS

Contratos y robustez: 29 casos adicionales

Grupo de casos	Esperado / obtenido	Evaluación
ID no entero en detalle, cupos, edición, cierre y eliminación	422 en los cinco casos.	PASS
Detalle/cupos inexistentes y ruta buscar	404 / 404 / 200. /buscar no se interpreta como un ID.	PASS
POST JSON y archivos inválidos	JSON no cumple el formulario: 422. No imagen y GIF: 422. Más de 5 MB: 413. PNG válido: 201.	PASS
Rango de precio inválido	422 antes de consultar.	PASS
Autocompletado	503 sin clave; 422 texto corto; 502 proveedor simulado fallido; blanco devuelve 200 y [].	PASS
Campos antiguos NULL en ambos listados	Precio, título y cupos: 6 casos devuelven 500 en vez de tolerar o tratar explícitamente la fila.	6 FAIL
Perfil inexistente	Se esperaba 404; devuelve 500.	1 FAIL
Foto de perfil no imagen	Se esperaba rechazo 422; acepta 16 bytes de texto y responde 200.	1 FAIL
Foto de perfil mayor a 5 MB	Se esperaba rechazo 413; responde 200 y truncaría el archivo a 5242880 bytes.	1 FAIL

Causas de los resultados negativos

Datos antiguos: main/main.py:267 convierte precio a float y el contrato de respuesta exige varios campos no nulos. No se comprobó que existan esas filas en datos reales: se reprodujo el comportamiento con registros simulados.

Perfil inexistente: obtener_perfil genera una HTTPException 404, pero su except Exception la transforma en 500. **Foto de perfil:** actualizar_perfil lee solo 5 MB y no reutiliza la comprobación de contenido de preparar_imagen.

El requisito de rechazar una foto de perfil inválida o grande es una expectativa recomendada de integridad, coherente con publicaciones; no es una respuesta de perfil que Swagger prometa actualmente. Todos los archivos se simularon, sin escribir imágenes reales.

06 / PRUEBAS Y EVIDENCIAS

Swagger: lo correcto y las seis brechas

Se auditaron **15 operaciones de /api y /auth**. El router de conexiones se aisló, por lo que su documentación y permisos no se consideran certificados por este informe. OpenAPI entrega versión 3.1.0; la API principal conserva metadatos genéricos FastAPI / 0.1.0.

Comprobaciones correctas

• /docs, /redoc, /openapi.json y /docs/oauth2-redirect responden 200.

• Publicar, listar y buscar tienen tags y respuesta de éxito tipada; publicar declara 201.

• Los campos obligatorios, límites de texto/cupos, enums principales y nueve filtros aparecen. Los operationId son únicos.

Criterio FAIL	Resultado observado / corrección requerida
Enum de mascota	Aparece como string libre. Documentar si/no y las reglas condicionales de mascotas.
Seguridad	No hay securitySchemes ni requisitos security. Primero verificar sesión en el backend y después reflejarla en el contrato.
Respuestas de éxito	12 de 15 operaciones muestran schema {}. Definir modelos para detalle, cupos, propias, edición/cierre, perfil y auth.
Estados de error	Faltan respuestas reales como 401, 404, 413 y 500; registro también usa 400/409. Añadirlas según cada operación.
Contenido del formulario	Solo declara multipart/form-data. También acepta URL encoded sin imagen y el README lo utiliza.
Forma de error 422	El esquema declara detail como array, pero ciertas reglas devuelven un string. Estandarizarlo o documentar ambas variantes.

Ejemplo de la diferencia en 422

```
Filtro tipo=oficina -> 422; detail es un array
Precio mínimo mayor al máximo -> 422; detail es un string
Mascota inválida -> 422; detail es un string
```

La auditoría de documentación tiene 20 PASS, 6 FAIL y 2 INFO. Los errores 422 se ejecutaron para comprobar la diferencia; no se dedujo solo por leer decoradores.

07 / PRUEBAS Y EVIDENCIAS

HTTP real local y guía manual

Se levantó Uvicorn en un puerto temporal de 127.0.0.1. Esta vez hubo conexiones TCP reales para los documentos y estáticos; PostgreSQL permaneció bloqueado y el ciclo de arranque de la aplicación se desactivó de forma explícita. El servidor temporal se detuvo al terminar.

Comprobación HTTP	Resultado
/docs	200 y marcador SwaggerUIBundle.
/redoc	200 y componente redoc.
/openapi.json	200 y contenido OpenAPI.
/docs/oauth2-redirect	200 y contenido del redirect.
/inicio y /crear	Ambas 200; contienen sus formularios reales.
/static_principal/HayCupo.js	200 y contenido JavaScript esperado.
/static_principal/tailwind.css	200 y contenido CSS esperado.
Bundle Swagger, CSS Swagger y bundle ReDoc	Los tres recursos públicos del CDN responden 200 y entregan datos.

11 de 11 PASS. Esto demuestra entrega HTTP y disponibilidad de los assets en el momento de la prueba; no certifica renderizado ni interacción en un navegador. No había un navegador conectado para revisar Try it out.

La guía manual está desactualizada

• La ruta de autenticación indicada en la guía no coincide con los archivos actuales.

• Indica cd main; la guía actual del backend requiere ejecutar main.main:app desde la raíz.

• MODO_PRUEBA_LOCAL y PUBLICACION_AUTOR_ID no son leídas por la dependencia de identidad actual.

• CP-PUB-06 espera un rechazo 503 al publicar sin autenticación. El código actual publica como usuario 1, por lo que ese control no debe marcarse como superado.

Referencia: taller de integra dos/endpoint/manual-testing-auth-publicaciones.md. Deben alinearse los pasos y resultados con el contrato corregido; una casilla de una guía no demuestra que una prueba se haya ejecutado.

08 / PRUEBAS Y EVIDENCIAS

Cómo repetirlo y qué corregir primero

Ejecutar desde la raíz del repositorio

```
.\.venv\Scripts\python.exe -B -m unittest discover -s tests -v
node tests/test_publicaciones_ui.cjs
.\.venv\Scripts\python.exe -B scripts/verificar_permisos_publicaciones.py
.\.venv\Scripts\python.exe -B scripts/verificar_contratos_publicaciones.py
.\.venv\Scripts\python.exe -B scripts/verificar_swagger_publicaciones.py
.\.venv\Scripts\python.exe -B scripts/verificar_http_swagger.py
```

Se necesitan las dependencias de main/requirements.txt y Node.js para la suite JavaScript. Los tres primeros verificadores nuevos devuelven código 1 mientras existan FAIL. Es un resultado intencional de los criterios incumplidos, no un error del PDF ni una razón para ocultar los fallos.

Prioridad	Acción / verificación de cierre
1. Identidad	Sesión verificable, sin respaldo al usuario 1. Anónimo y cookie forjada deben rechazarse; repetir el flujo después de logout.
2. Datos y perfil	Política de datos antiguos y contrato coherente; conservar HTTPException 404; validar contenido/tamaño de fotos sin truncarlas.
3. Swagger	Modelos de respuesta, estados de error, seguridad, enum/condiciones de mascotas, tipos de formulario y error 422 consistente.
4. Documentación y recorrido	Actualizar la guía manual; después ejecutar un recorrido de navegador y persistencia en una base de pruebas autorizada.

Límites de la entrega

No se usó PostgreSQL real ni se escribieron cuentas o publicaciones reales. No se probaron migraciones, arranque completo con la base, carga, despliegue ni interacción visual de Swagger. Las operaciones de conexiones quedaron fuera del alcance. Los JSON registran resultados esperados y obtenidos, y se actualizarán al repetir cada script.

El PDF es una fotografía de esta revisión y no se regenera automáticamente al volver a ejecutar los verificadores. Se conservaron los informes anteriores. Se añadieron scripts QA, evidencias y este informe; no se aplicaron correcciones a la lógica de producto.

Conclusión: la ejecución pedida está terminada y es reproducible. Hay evidencia positiva del funcionamiento normal y evidencia negativa de controles que todavía no cumplen la política de seguridad o robustez. El siguiente paso es corregir esos controles y volver a ejecutar las mismas expectativas.